

PYTHON ESSENTIALS FOR ODOO DEVELOPMENT

Python for Odoo Developers

Core Python concepts used every day in Odoo

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

July 2026

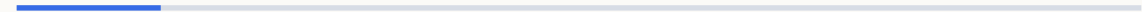


Agenda

- 1 Section 1: Running Python programs — 4 topics
- 2 Section 2: Basic values and text — 10 topics
- 3 Section 3: Collections of data — 9 topics
- 4 Section 4: Control flow — 8 topics
- 5 Section 5: Functions and reusable code — 7 topics
- 6 Section 6: Objects, errors, modules, and Odoo habits — 11 topics

1 Section 1

Running Python programs



1.1 What Python and the interpreter are

- Python is a programming language used to write instructions for a computer.
- The interpreter is the program that reads and runs Python code.
- Odoo server code, automation scripts, and shell experiments all run through Python.

Check interpreter path

```
python3 -c "import sys; print(sys.executable)"
```

Run one instruction

```
python3 -c "print('Hello from Python')"
```

1.2 Run python and python3

- On many Linux systems, python3 is the command for modern Python.
- The python command may be missing or may point to a different version.
- Check the version when a server has more than one Python installed.

Check Python 3 version

```
python3 --version
```

Compare command versions

```
python --version || true
```

1.3 First program

- A Python program is a text file containing Python statements.
- The print function displays output in the terminal.
- Run the file again after each small change to build confidence.

Create hello.py

```
print('Hello, Merit Advisory')
```

Run the file

```
python3 hello.py
```

1.4 Python 2 vs Python 3

- Python 3 is the standard for current Odoo development.
- Old examples on the internet may use Python 2 syntax.
- Version mismatches often appear as syntax errors or missing library behavior.

Modern print syntax

```
print('Odoo uses Python 3')
```

Show exact runtime

```
python3 -c "import sys; print(sys.version)"
```

2 Section 2

Basic values and text

2.1 Numbers

- Python supports integers for whole numbers and floats for decimal values.
- Arithmetic operators include plus, minus, multiply, divide, and exponent.
- Use parentheses when you want to make calculation order obvious.

Calculate subtotal

```
subtotal = 100 + 25  
print(subtotal)
```

Calculate tax

```
tax = 125 * 0.15  
print(tax)
```

2.2 Variables

- A variable is a name bound to a value.
- Choose names that explain what the value represents.
- Python variables do not need a type written in front of the name.

Store a customer name

```
customer_name = 'Agrolait'  
print(customer_name)
```

Update a count

```
invoice_count = 3  
invoice_count = invoice_count + 1  
print(invoice_count)
```

2.3 Strings

- A string is text surrounded by quotes.
- Single quotes and double quotes both work when used consistently.
- Strings are common for names, descriptions, XML ids, and messages.

Create a string

```
module_name = 'sale_custom'  
print(module_name)
```

Use double quotes

```
message = "Invoice posted"  
print(message)
```

2.4 String concatenation

- Concatenation joins strings together with the plus operator.
- Both sides must be strings or Python raises a type error.
- For larger messages, f-strings are usually easier to read.

Join two strings

```
name = 'Odoo'  
print('Hello ' + name)
```

Convert before joining

```
count = 3  
print('Invoices: ' + str(count))
```

2.5 f-strings

- An f-string starts with f before the quote.
- Expressions inside braces are evaluated and inserted into the text.
- Use f-strings for clear messages that include values.

Format a customer message

```
partner = 'Deco Addict'  
print(f'Customer: {partner}')
```

Format a total

```
total = 149.99  
print(f'Total due: {total}')
```

2.6 String indexes

- Indexes select one character from a string.
- Python indexes start at zero.
- Negative indexes count backward from the end.

First character

```
code = 'INV001'  
print(code[0])
```

Last character

```
code = 'INV001'  
print(code[-1])
```

2.7 String slicing

- A slice extracts part of a string.
- The start index is included and the stop index is not included.
- Slicing is useful for prefixes, suffixes, and fixed-format codes.

Get prefix

```
number = 'INV/2026/001'  
print(number[:3])
```

Get year

```
number = 'INV/2026/001'  
print(number[4:8])
```

2.8 String immutability

- Strings are immutable, so their characters cannot be changed in place.
- String methods return new strings instead of editing the original.
- Assign the result when you want to keep the changed value.

Create a changed copy

```
name = 'odoo'  
upper_name = name.upper()  
print(upper_name)
```

Replace text

```
code = 'S0001'  
code = code.replace('S0', 'INV')  
print(code)
```


2.9 Type conversion

- Type conversion creates a value of another type.
- Common conversion functions include str, int, float, and bool.
- Convert input data before doing math or comparisons.

Convert text to integer

```
quantity = int('5')  
print(quantity + 2)
```

Convert number to text

```
order_id = 42  
print('Order ' + str(order_id))
```

2.10 Booleans

- A boolean is either True or False.
- Booleans are used for decisions and status flags.
- Comparison expressions create boolean results.

Store active state

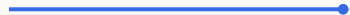
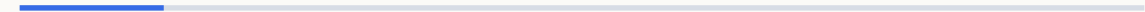
```
is_active = True  
print(is_active)
```

Compare amount

```
amount = 250  
print(amount > 100)
```

3 Section 3

Collections of data



3.1 Lists

- A list stores multiple values in order.
- Lists can grow, shrink, and be changed after creation.
- Use lists when order matters or when you need a sequence of records.

Create a list

```
partners = ['Agrolait', 'Azure Interior']  
print(partners)
```

Read by index

```
partners = ['Agrolait', 'Azure Interior']  
print(partners[0])
```

3.2 List slicing

- List slicing returns a new list containing part of another list.
- The start index is included and the stop index is not included.
- Slicing is useful for paging, previews, and small batches.

First two items

```
ids = [10, 11, 12, 13]  
print(ids[:2])
```

Middle items

```
ids = [10, 11, 12, 13]  
print(ids[1:3])
```

3.3 append - Add to a list

- append adds one item to the end of a list.
- It changes the existing list in place.
- Use append when building a collection step by step.

Add one customer

```
partners = []  
partners.append('Agrolait')  
print(partners)
```

Collect ids

```
record_ids = [1]  
record_ids.append(2)  
print(record_ids)
```

3.4 pop - Remove from a list

- pop removes an item from a list and returns it.
- Without an index, pop removes the last item.
- Use pop when the removed value still needs to be used.

Remove last item

```
tasks = ['check', 'restart']  
next_task = tasks.pop()  
print(next_task)
```

Remove first item

```
tasks = ['check', 'restart']  
first_task = tasks.pop(0)  
print(first_task)
```

3.5 Dictionaries

- A dictionary stores key and value pairs.
- Keys are used to look up related values.
- Dictionaries are common in Odoo values, context, and API payloads.

Create values

```
vals = {'name': 'Agrolait', 'active': True}  
print(vals)
```

Read a value

```
vals = {'name': 'Agrolait'}  
print(vals['name'])
```


3.6 dict.get

- get reads a dictionary value without raising an error for a missing key.
- You can provide a default value as the second argument.
- Use get when missing data is expected and acceptable.

Get missing value

```
vals = {'name': 'Agrolait'}  
print(vals.get('email'))
```

Get with default

```
vals = {'name': 'Agrolait'}  
print(vals.get('active', True))
```

3.7 dict.keys

- keys shows the keys currently stored in a dictionary.
- It is useful when inspecting unfamiliar data.
- Convert to list when you need a simple printable list of keys.

Show keys

```
vals = {'name': 'Agrolait', 'active': True}
print(vals.keys())
```

Print key list

```
vals = {'name': 'Agrolait'}
print(list(vals.keys()))
```

3.8 Tuples

- A tuple stores ordered values like a list.
- Tuples are immutable, so their contents cannot be changed in place.
- Use tuples for fixed pairs or values that should stay together.

Create a tuple

```
status = ('draft', 'Draft')  
print(status)
```

Unpack a tuple

```
code, label = ('posted', 'Posted')  
print(label)
```

3.9 Sets

- A set stores unique values with no guaranteed order.
- Adding a duplicate value keeps only one copy.
- Use sets when uniqueness matters more than order.

Remove duplicates

```
tags = {'vip', 'vip', 'retail'}  
print(tags)
```

Find shared items

```
a = {'sale', 'stock'}  
b = {'stock', 'account'}  
print(a & b)
```

4 Section 4

Control flow

4.1 if, elif, and else

- if runs code only when a condition is true.
- elif checks another condition when the previous one was false.
- else handles the remaining case.

Check amount

```
amount = 120
if amount > 100:
    print('large order')
```

Choose status message

```
state = 'draft'
if state == 'posted':
    print('done')
elif state == 'draft':
    print('needs review')
else:
    # ...
```

4.2 Truthy and falsey values

- Python treats some non-boolean values as true or false in conditions.
- Empty strings, empty lists, empty dictionaries, zero, and None are falsey.
- Non-empty values are usually truthy.

Check a missing email

```
email = ''  
if not email:  
    print('email missing')
```

Check a list

```
invoices = [101]  
if invoices:  
    print('has invoices')
```

4.3 for loops

- A for loop repeats code for each item in a sequence.
- The loop variable receives one item at a time.
- Use for loops to process lists of records or values.

Print names

```
for name in ['Agrolait', 'Deco Addict']:  
    print(name)
```

Total amounts

```
total = 0  
for amount in [10, 20, 30]:  
    total += amount  
print(total)
```


4.4 range

- range creates a sequence of numbers for looping.
- The stop value is not included.
- Use range when the number of repetitions matters.

Loop three times

```
for i in range(3):  
    print(i)
```

Start and stop

```
for number in range(1, 4):  
    print(number)
```

4.5 enumerate

- enumerate gives both the index and the item during a loop.
- It avoids manually updating a counter.
- Use it when position and value are both useful.

Number names

```
names = ['Agrolait', 'Deco Addict']  
for index, name in enumerate(names):  
    print(index, name)
```

Start at one

```
lines = ['first', 'second']  
for number, line in enumerate(lines, start=1):  
    print(number, line)
```

4.6 while loops

- A while loop repeats while a condition stays true.
- The loop must change something or it may run forever.
- Use while loops for retry logic or waiting conditions.

Count upward

```
count = 0
while count < 3:
    print(count)
    count += 1
```

Process queue

```
queue = ['a', 'b']
while queue:
    print(queue.pop(0))
```

4.7 break

- break exits the nearest loop immediately.
- Use it when the work is finished before the loop naturally ends.
- It is common when searching for the first matching value.

Stop at match

```
for name in ['A', 'B', 'C']:
    if name == 'B':
        break
    print(name)
```

Find first large amount

```
for amount in [20, 150, 10]:
    if amount > 100:
        print(amount)
        break
```

4.8 continue

- continue skips the rest of the current loop step.
- The loop then moves to the next item.
- Use it to skip values that should not be processed.

Skip empty names

```
for name in ['Agrolait', '', 'Deco']:
    if not name:
        continue
    print(name)
```

Skip draft records

```
for state in ['draft', 'posted']:
    if state == 'draft':
        continue
    print(state)
```

5 Section 5

Functions and reusable code

5.1 Functions

- A function is a named block of reusable code.
- Use def to create a function.
- Functions make programs easier to read, test, and reuse.

Define and call

```
def greet():  
    print('Hello')  
  
greet()
```

Reusable action

```
def show_ready():  
    print('Ready for Odoo work')  
  
show_ready()
```

5.2 Parameters

- Parameters are names that receive values passed into a function.
- They let one function work with different input values.
- Use clear parameter names to describe expected data.

Pass a name

```
def greet(name):  
    print(f'Hello {name}')
```



```
greet('Agrolait')
```

Pass an amount

```
def show_total(amount):  
    print(f'Total: {amount}')
```



```
show_total(250)
```


5.3 Default parameters

- A default parameter value is used when no argument is provided.
- Defaults make common calls shorter.
- Avoid mutable defaults such as empty lists unless you understand the behavior.

Default greeting

```
def greet(name='team'):  
    print(f'Hello {name}')
```

`greet()`

Override default

```
def limit_text(text, limit=10):  
    print(text[:limit])
```

`limit_text('Odo training', 4)`

5.4 return

- return sends a result back to the caller.
- Code after return in the same function is not executed.
- Use return when another part of the program needs the computed value.

Return a total

```
def add_tax(amount):  
    return amount * 1.15  
  
print(add_tax(100))
```

Return a message

```
def invoice_label(number):  
    return f'Invoice {number}'  
  
print(invoice_label('INV001'))
```

5.5 *args

- *args collects extra positional arguments into a tuple.
- Use it when a function may receive a flexible number of values.
- Keep it simple for beginners and prefer named parameters when possible.

Sum many amounts

```
def total(*amounts):  
    return sum(amounts)  
  
print(total(10, 20, 30))
```

Print labels

```
def show_labels(*labels):  
    for label in labels:  
        print(label)  
  
show_labels('draft', 'posted')
```

5.6 **kwargs

- **kwargs collects extra named arguments into a dictionary.
- It is common in framework code that passes flexible options.
- Read keyword names carefully because misspellings can hide bugs.

Collect named values

```
def show_values(**kwargs):  
    print(kwargs)  
  
show_values(name='Agrolait', active=True)
```

Read an option

```
def connect(**options):  
    print(options.get('host', 'localhost'))  
  
connect(host='db01')
```

5.7 Scope

- Scope describes where a name can be used.
- Names created inside a function are local to that function.
- Pass values into functions instead of relying on hidden outside state.

Local variable

```
def build_name():  
    name = 'Agrolait'  
    return name  
  
print(build_name())
```

Pass outside value

```
prefix = 'INV'  
def make_number(number):  
    return f'{prefix}{number}'  
  
print(make_number('001'))
```

6 Section 6

Objects, errors, modules, and Odoo habits

6.1 Classes and `__init__`

- A class defines a kind of object with data and behavior.
- `__init__` runs when a new object is created.
- `self` refers to the current object inside the class.

Create a class

```
class Partner:
    def __init__(self, name):
        self.name = name

partner = Partner('Agrolait')
print(partner.name)
```

Store multiple fields

```
class Invoice:
    def __init__(self, number, amount):
        self.number = number
        self.amount = amount

print(Invoice('INV001', 100).amount)
```

6.2 Methods

- A method is a function defined inside a class.
- Methods use self to read or change object data.
- Odoo models are classes with many framework methods.

Call a method

```
class Partner:
    def greet(self):
        return 'Hello'

print(Partner().greet())
```

Use object data

```
class Partner:
    def __init__(self, name):
        self.name = name
    def label(self):
        return f'Partner: {self.name}'

# ...
```


6.3 Inheritance

- Inheritance lets one class reuse behavior from another class.
- The child class can add or replace methods.
- Odoo module customization often uses inheritance to extend existing models.

Create child class

```
class Document:
    def label(self):
        return 'Document'

class Invoice(Document):
    pass
# ...
```

Override method

```
class Document:
    def label(self):
        return 'Document'

class Invoice(Document):
    def label(self):
        # ...
```

6.4 super

- super calls behavior from a parent class.
- Use it when extending a method instead of replacing everything.
- In Odoo, forgetting super can skip important framework behavior.

Extend parent method

```
class Base:
    def label(self):
        return 'Base'

class Child(Base):
    def label(self):
        # ...
```

Extend setup

```
class Base:
    def __init__(self):
        self.ready = True

class Child(Base):
    def __init__(self):
        # ...
```

6.5 try and except

- try runs code that may fail.
- except handles a specific error so the program can respond.
- Catch expected errors and avoid hiding problems silently.

Handle bad number

```
try:  
    quantity = int('five')  
except ValueError:  
    print('quantity must be a number')
```

Handle missing key

```
vals = {}  
try:  
    print(vals['name'])  
except KeyError:  
    print('name is missing')
```

6.6 Modules and import

- A module is a Python file that can be imported by other code.
- import loads names from modules so you can reuse existing functionality.
- Odoo addons are organized as Python packages and modules.

Import a standard module

```
import datetime  
print(datetime.date.today())
```

Import one name

```
from pathlib import Path  
print(Path.cwd())
```

6.7 Virtual environments

- A virtual environment keeps Python packages separate for one project.
- It helps avoid conflicts between system packages and project packages.
- Activate the environment before installing or running project code.

Create environment

```
python3 -m venv .venv
```

Activate environment

```
source .venv/bin/activate
```

6.8 pip install

- pip installs Python packages into the active environment. **Install a package**
- Use `python -m pip` so pip belongs to the interpreter you are using.
- For projects, record dependencies in a requirements file. **Install requirements**

```
python3 -m pip install requests
```

```
python3 -m pip install -r requirements.txt
```

6.9 Reading Python errors

- Python tracebacks show where an error happened and what type it was.
- Read from the bottom for the final error message.
- Then scan upward to find your code path that led to the error.

Cause a NameError

```
print(missing_name)
```

Cause a TypeError

```
print('Total: ' + 10)
```

6.10 Debugging basics

- Debugging means finding why actual behavior differs from expected behavior.
- Print small facts such as type, value, and branch taken.
- Change one thing at a time so you know what affected the result.

Print value and type

```
amount = '100'  
print(amount, type(amount))
```

Use breakpoint

```
amount = 100  
breakpoint()  
print(amount * 2)
```


6.11 Python habits for Odoo

- Write small readable methods that match Odoo model responsibilities.
- Prefer clear names for recordsets, dictionaries, and computed values.
- Restart, upgrade, and read logs when testing server-side code changes.
- Keep framework behavior in mind before overriding methods.

Name values clearly

```
partner_vals = {'name': 'Agrolait', 'customer_rank': 1}
```

Use a readable helper

```
def is_large_invoice(amount):  
    return amount >= 1000
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory
02 · Python

